Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow

Joint placement and per-request scheduling

Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang,
Zhihao Jia, Rashmi Vinayak
Carnegie Mellon University — ASPLOS 2025

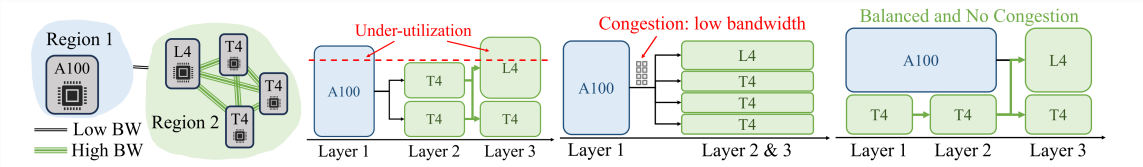
Research meeting: 4 November 2026
Yu-Jui Liang Advisor: Hsiang-Yun Cheng
National Taiwan University / Academia Sinica

Source: Helix, title page

Discussion Index

Why Do We Need Helix?

Mixed GPU speeds and memory capacities interact with slow inter-region links.



Equal layer counts Slow stages limit the whole pipeline.

Balanced computation Every request can still cross a congested network cut.

Joint placement and routing Local execution can reduce traffic across that cut.

Source: Helix, Secs. 3–4.2, Fig. 1 (illustrative cluster)

Motivation Details

What Does Helix Optimize?

Placement Ψ Joint MILP chooses contiguous layer ranges for GPU nodes under memory constraints.

Capacity $c_{uv}(\Psi)$ GPU profiling, network bandwidth, and placement determine valid graph edges and capacity limits.

Flow f_{uv} MILP jointly optimizes flow allocation with placement to maximize aggregate serving throughput.

$$\max_{\Psi, f} \sum_{i} f_{\text{source},i} \qquad \text{subject to} \quad 0 \leq f_{uv} \leq c_{uv}(\Psi)$$

Serving throughput in tokens/s

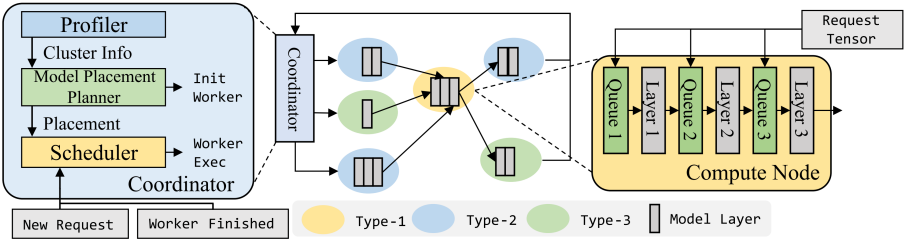
The throughput objective does not directly optimize latency or its percentiles:

TTFT: Time from request submission to the first token.
TPOT: Average time per output token after the first.
P95 / P99 latency: Latency thresholds exceeded by the slowest 5% / 1% of requests.

Source: Helix, Secs. 4.3–4.4, 5.1

Cost Model MILP Variables

How Does One Request Run?



Offline planning Profile GPU / network resources, jointly solve placement and flow, and load model weights.

Online scheduling On arrival, the coordinator checks estimated KV capacity and uses optimized flow weights in IWRR to select one valid pipeline.

Pipeline execution Workers execute assigned layers in order, forward activations, and return the output token to the coordinator.

Decode reuse Later tokens reuse the request's pipeline and local KV caches. MILP is not rerun per request or token.

Source: Helix, Secs. 4–5, Fig. 3; lifecycle synthesis

Runtime Details KV Cache

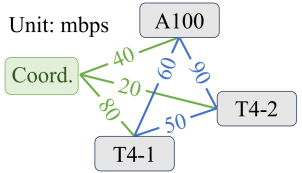
How Does Max Flow Model Serving?

Node splitting $c^{in} \to c^{out}$ limits compute throughput.

Network edges Bandwidth / payload gives tokens/s.

Source and sink The coordinator injects and collects tokens.

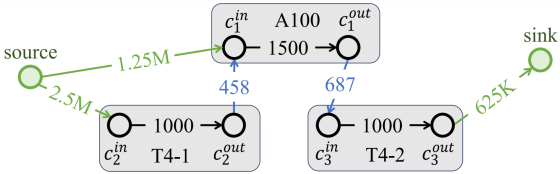
Conservation Flow enters and leaves each compute node at the same rate.



	Id	Holding
A100	c_1	Layer 1 & 2
T4-1	c_2	Layer 1
T4-2	c_3	Layer 3

Token size: 4 Byte
Activation size: 16 KB

(a) Physical links and layer placement



(b) Capacities in tokens/s

For fixed Ψ, $F(\Psi) = \text{MaxFlow}(G(\Psi))$ models steady-state throughput.
The next slide jointly optimizes placement and flow.

Source: Helix, Sec. 4.3, Fig. 2

Worked Cost Model

How Does MILP Decide Placement?

$$\max \sum_i f_{\text{source},i}$$

Find the placement that supports the highest aggregate serving throughput, jointly optimizing placement and flow.

1. Placement and memory Assign each GPU a contiguous layer range that fits its memory budget.

2. Flow conservation Keep incoming and outgoing token rates equal at every worker.

3. Compute capacity Limit each GPU's token rate to its profiled processing capacity.

4. Connectivity Allow only hops that continue through model layers in order, from the first layer to the last.

5. Network capacity Keep traffic on each valid link within its bandwidth-based token rate.

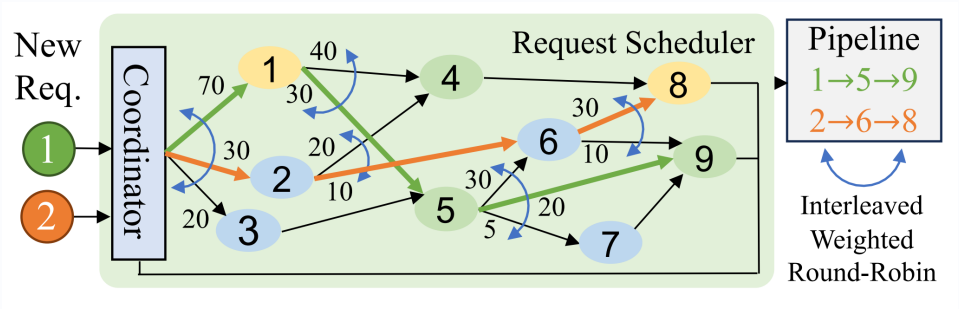
Pruning and timed solving produce feasible plans without always proving global optimality.

Source: Helix, Secs. 4.4–4.6, 6.2

MILP Variables Solver Settings

How Does Runtime Scheduling Work?

IWRR = interleaved weighted round-robin. All path selection runs at the coordinator.



Weights Use optimized edge flows, rather than raw link capacities.

Selection Walk the valid graph using each vertex's IWRR state, after KV masking.

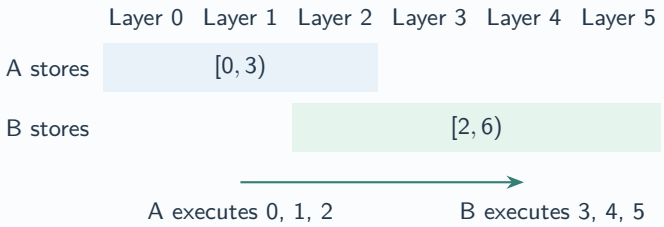
Reuse Interleaving avoids bursts. A request reuses its pipeline across decode steps.

Source: Helix, Sec. 5.1, Fig. 4

IWRR Scheduling

How Are Weights and KV Cache Managed?

Illustrative example: six layers, numbered 0–5. Half-open ranges.



Overlapping weights B stores layer 2, but this request skips it on B.

KV ownership Keep request-specific KV for the layers actually executed on each worker.

KV-aware selection Mask workers whose estimated KV usage exceeds the high-water mark, then apply flow-weighted IWRR to the remaining candidates.

Source: Helix, Secs. 4.4, 5.1–5.2, 6.1; illustrative diagram

Partial Inference | KV Cache

How Is Helix Evaluated?

Setting	Hardware / network	Evidence
Single cluster	4 A100 + 8 L4 + 12 T4; 10 Gb/s	24-node prototype + simulator validation
Geo-distributed	Same 24 nodes, 3 clusters; mean 100 Mb/s, 50 ms across clusters	Simulation
Higher heterogeneity	42 nodes, 7 types including multi-GPU nodes	Simulation

Models and workload FP16 LLaMA-1 30B / LLaMA-2 70B; filtered Azure Conversation trace.

Remove requests with input > 2048 or output > 1024 tokens.
Retain 16,657 requests: mean input 763 tokens, mean output 232 tokens.

Metrics Offline saturation: decode tokens/s. Online: throughput plus prompt / decode latency.

Source: Helix, Secs. 6.1–6.2

Experiment Setup Workload

What Does Helix Compare Against?

Method	Core approach	Comparison scope
Helix	Joint MILP placement and flow; flow-weighted IWRR	Jointly accounts for GPU and network limits.
Swarm	Heuristic placement and scheduling adapted from heterogeneous training	End-to-end serving baseline, reimplemented in Helix.
SP	Separate model pipelines for each GPU type	End-to-end serving baseline; restricts mixing GPU types within a pipeline.
Petals	Decentralized placement heuristic	Placement-only baseline; no end-to-end serving comparison.

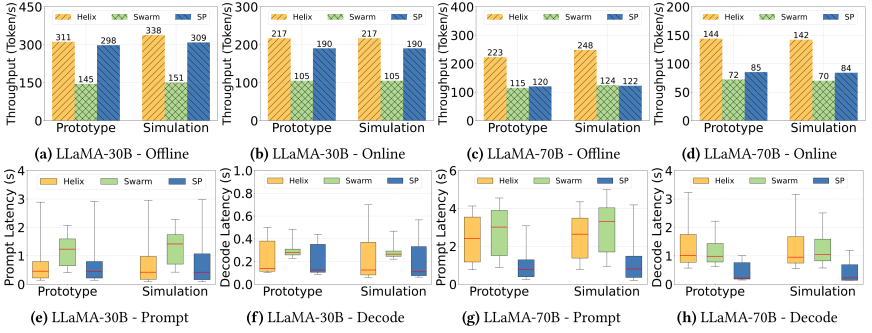
Placement ablation: Fix the Helix scheduler and vary placement (Helix / Swarm / Petals) to isolate placement gains.

Source: Helix, Secs. 2.2, 6.2, 6.6, Fig. 9

Baseline Details Placement Ablation

Does Helix Improve Single-Cluster Serving?

24-node prototype and simulator check. FP16 30B / 70B. Baselines: SP and Swarm.



(a) LLaMA-30B - Offline **(b)** LLaMA-30B - Online **(c)** LLaMA-70B - Offline **(d)** LLaMA-70B - Online

(e) LLaMA-30B - Prompt **(f)** LLaMA-30B - Decode **(g)** LLaMA-70B - Prompt **(h)** LLaMA-70B - Decode

Figure 6. Single cluster results: (a - d) decode throughput, **(e - h)** prompt and decode latency for online serving. The box shows the 25 and 75 percentile. The whisker shows the 5 and 95 percentile. The red line shows median.

Offline: Supply enough requests to saturate the cluster; measure achievable throughput.

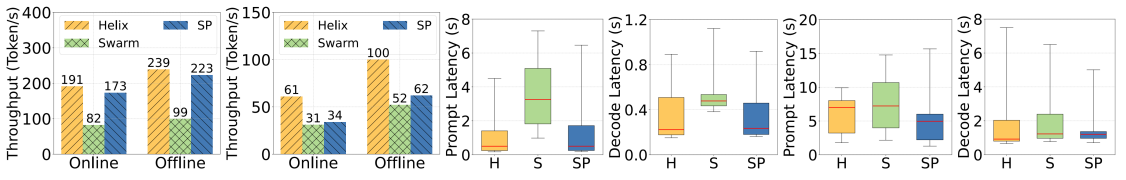
Online: Azure Conversation arrivals average 75% of peak throughput; the cluster is not always saturated.

Source: Helix, Secs. 6.2–6.3, Fig. 6

All Ratios Full Figure

Does Helix Work Across Regions?

Simulation: 24 nodes in 3 clusters; mean inter-cluster 100 Mb/s and 50 ms.



(a) LLaMA-30B **(b)** LLaMA-70B **(c)** LLaMA-30B **(d)** LLaMA-30B **(e)** LLaMA-70B **(f)** LLaMA-70B

Figure 7. Geo-distributed clusters: (a - b) decode throughput, **(c - f)** prompt and decode latency for online serving. "H" stands for Helix, "S" stands for Swarm. Box: 25 and 75 percentile. Whisker: 5 and 95 percentile. Red line: median.

Throughput Helix achieves higher throughput than both baselines across regions, with larger gains over SP for the larger model.

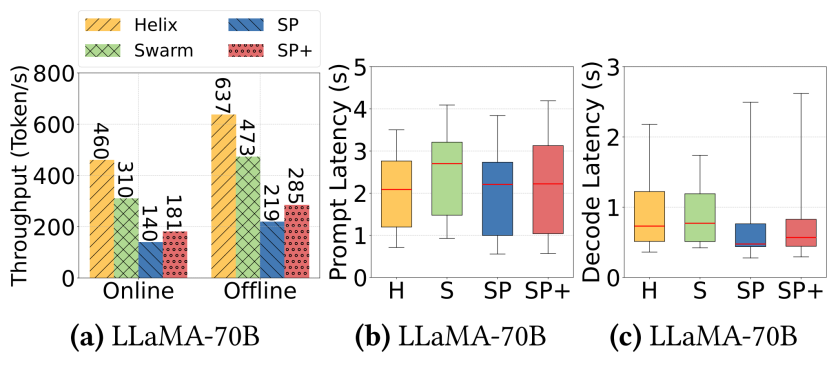
Latency Helix improves latency over Swarm, but does not consistently outperform both baselines.

Source: Helix, Sec. 6.4, Fig. 7

Geo Details Full Figure

What Happens with More GPU Types?

42-node simulation; LLaMA-2 70B FP16; Swarm, SP, and SP+ baselines.



(a) LLaMA-70B **(b)** LLaMA-70B **(c)** LLaMA-70B

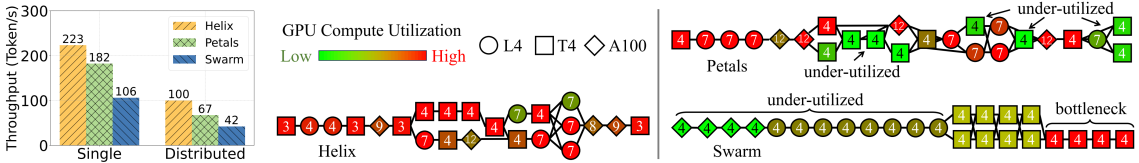
Helix maintains higher throughput with greater GPU heterogeneity, but does not consistently improve latency.

Source: Helix, Sec. 6.5, Fig. 8

42-node Details

Where Do the Gains Come From: Placement?

Offline LLaMA-2 70B. Fixed Helix scheduler; vary placement only.



(a) Decode throughput.

(b) Model placement case study. Numbers on each node represent the number of layers the node holds.

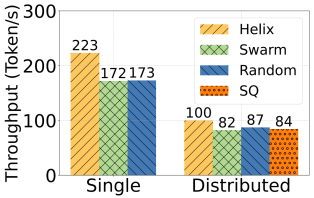
With the same Helix scheduler, Helix's placement achieves the highest throughput and keeps GPUs more fully utilized.

Source: Helix, Sec. 6.6, Fig. 9

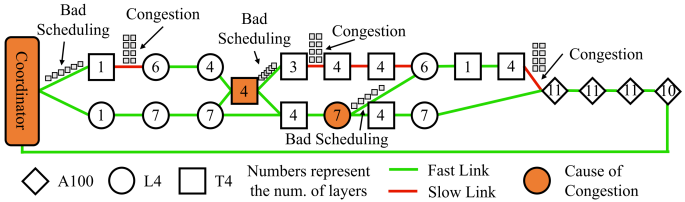
Ablation Details Graph Detail

Where Do the Gains Come From: Scheduling?

Offline LLaMA-2 70B. Fixed Helix placement; vary scheduler only.



(a) Decode throughput.



(b) Request scheduling case study.

MILP placement and scheduling work together: even with optimized placement, Helix's flow-aware IWRR scheduler reduces network congestion and further improves throughput.

Source: Helix, Sec. 6.7, Fig. 10

Ablation Details Graph Detail

Conclusion & Discussion

Contribution — Joint Placement and Scheduling

Max-flow models heterogeneous GPU compute and network capacities. MILP jointly optimizes layer placement and flow; IWRR maps the solution to per-request pipelines.

Evidence — Both Components Improve Throughput

- 24-node prototype: 1.69× online throughput vs. SP (70B).
- Geo-distributed and 42-node evaluations use simulation.
- Ablations confirm gains from both placement and scheduling.

Trade-off — Throughput vs. Latency

Better utilization of slower GPUs improves aggregate throughput, but may increase decode latency.

Limitations — Static Planning

Offline placement and profiled capacities; average-length KV estimates; finite MILP solving budget.

Research Questions

- When should placement and flow weights adapt to workload or network changes?
- How can throughput optimization incorporate latency SLOs and power constraints?

Source: Helix, Secs. 4–6, Figs. 6–10. Research questions are proposed extensions.

Discussion Index Research Directions

Discussion Index

Topic	Questions	Jump
A. Background	GPU heterogeneity, TP / PP / DP, placement bottlenecks	Open A
B. Max flow / MILP	Cost units, constraints, partial inference, solver	Open B
C. Runtime	Architecture, IWRR, KV cache, batching	Open C
D. Evidence	Settings, trace, complete figures, all ratios	Open D
E. Discussion	Static plans, latency, validity, future work	Open E

Every backup has a numbered **Main** return link and related-topic links.

The footer opens this index from any page.

Main 16

Discussion Index

Why Heterogeneous and Geo-Distributed Serving?

Table 3. Datasheet properties. SXM for H100 and A100. Not measured serving throughput.

GPU	FP16 (TFLOP/s)	Memory	Bandwidth	Price (USD)
H100	1979	80 GB	3350 GB/s	25k–40k
A100	312	40 GB	1555 GB/s	10k–15k
L4	242	24 GB	300 GB/s	∼3k
T4	65	16 GB	300 GB/s	∼1k

- Table 1: if half the memory stores parameters, LLaMA-2 70B needs 12 L4s, 7 A100s, or 4 H100s.
- Table 4: in one us-east window the quota was 8 A100 + 8 L4 + 16 T4, but only 16 T4s were deployed.
- The paper's claim: eight L4s have FP16 throughput comparable to one H100, with more memory and less power.

Scarcity, not a universal speed ranking, is why Helix aggregates mixed GPUs across regions.

Source: Helix, Sec. 1, Tables 1–4. Eight-L4 comparison is a datasheet argument.

Main 2 Parallelism Placement Network Discussion Index

Parallelism Across Nodes and GPUs

Tensor parallelism (TP) Split a layer across GPUs inside one node. Each layer requires two collectives, making high-latency cross-node TP expensive.

Pipeline parallelism (PP) Partition layers across nodes and transfer activations between stages. Equal cuts can leave faster GPUs waiting.

Data parallelism (DP) Replicas process different requests. A replica may itself use TP / PP.

Petals A joining node greedily takes the least-loaded stage in a decentralized volunteer system.

Swarm Even model partitions and next-stage routing using real-time throughput. Helix reimplements this training method as an inference baseline.

Source: Helix, Secs. 2.2–3, 6.2

Main 2 Discussion Index

Uniform Model Placement Is Sub-Optimal

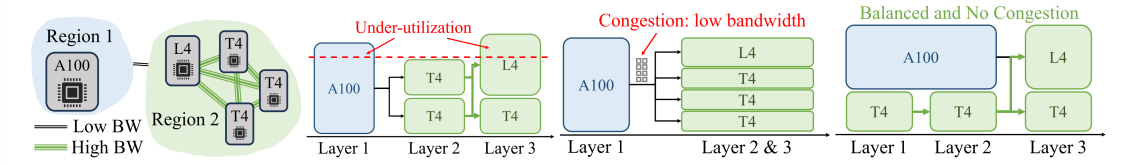


Fig. 1a–c. Illustrative example, not the 24-node cluster. Order: A100 > L4 > T4.

(b) Equal cuts still wait on the slower stage. (c) Balanced FLOPs still cross the slow inter-region link on every request.

Source: Helix, Sec. 3.1 and Sec. 4.2, Fig. 1a–c. PDF p. 4.

Main 2 | Network Congestion | Discussion Index

Network-Unaware Routing Creates Congestion

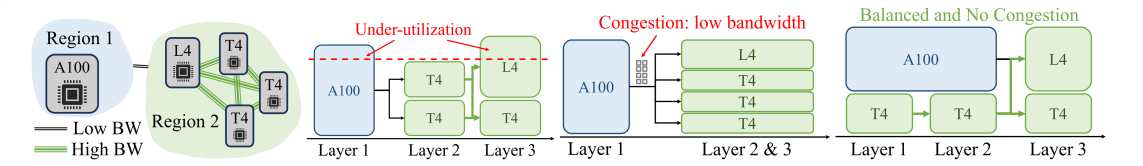


Fig. 1c–d. Same illustrative cluster as the previous slide.

- In (c) the A100's send side congests: every activation still crosses into Region 2.
- (d) runs two T4s beside the A100, so less traffic uses the slow link.
- Partition, placement, and routing have to be chosen together.

Source: Helix, Sec. 4.2, Fig. 1c–d. PDF p. 4.

Main 2

Discussion Index

Helix Design Requirements

Placement One cluster plan assigns each node a contiguous range of layers, respecting memory and the network.

Throughput estimate Measure the token rate that can flow from the coordinator through workers and back.

Scheduling Choose a pipeline per request. Different requests can share workers instead of using disjoint fixed pipelines.

Offline placement and online scheduling share the graph but run at different timescales.

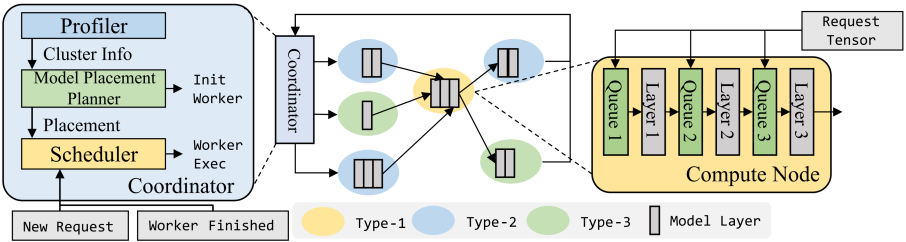
KV-cache admission is a runtime mask. Table 6 does not add a separate KV-byte objective or variable to the placement MILP.

Source: Helix, Secs. 3–5

Main 3

Discussion Index

Helix System Overview



- Profiler collects cluster info. The planner solves placement once per cluster.
- A new request gets a pipeline and is sent to the first node only.
- Each node runs its layers and forwards the activation.
- The last node returns the token. The next token reuses the same pipeline.

The coordinator owns placement and scheduling. Compute nodes do not re-plan the route.

Source: Helix, Sec. 5, Fig. 3. PDF p. 7.

Main 4

Discussion Index

From Model Placement to a Flow Network

- A cluster has one coordinator h and compute nodes C. A multi-GPU machine is one logical node: compute and VRAM are summed.
- Placement Ψ gives each node a contiguous layer range. Scheduling ϕ gives each request an ordered node sequence.
- A common score keeps only the slowest stage, $\min_i \sum_j \mathrm{capacity}(j) \cdot \mathbf{1}[i \in \Psi(j)]$.
- Fig. 1c has balanced stage FLOPs and is still limited by the wide-area link. The min-stage score cannot see that.

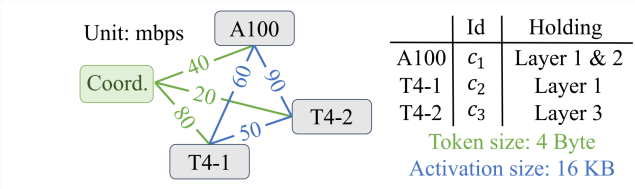
Helix scores a placement by the max flow of a graph, in tokens per second.

Source: Helix, Sec. 4.1–4.2. The min-stage formula is the paper's description of prior heuristics.

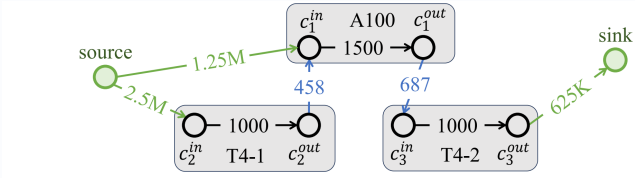
Main 5 | Main 3 | Design | Graph | Compute | Network | Throughput | Discussion Index

Max-Flow Graph Abstraction

- Each compute node becomes $c^{\text{in}} \to c^{\text{out}}$.
- The coordinator becomes the source and the sink.
- An edge capacity is how many tokens may cross it per second.
- Max flow from source to sink is the serving throughput of this placement.



(a) Physical links and layer placement



(b) Capacities in tokens/s

Fig. 2. Capacities are tokens/s.

Source: Helix, Sec. 4.3, Fig. 2. PDF p. 4.

Main 5 | Compute | Network | Throughput

Discussion Index

Modeling Compute Capacity

Split each worker $c^{in} \rightarrow c^{out}$ is the internal edge through which all assigned flow must pass.

$$c_{\text{node}} = \min\{\text{profiled compute tokens/s},\ \text{node network tokens/s}\}$$

Profile the layer count T_j is the measured token rate for a node holding j layers. MILP binaries select the appropriate rate.

Coordinator Source and sink inject and collect tokens, without a GPU compute edge.

Datasheet TFLOP/s motivate heterogeneity; measured token rates parameterize the serving model.

Source: Helix, Secs. 4.3–4.4, Constraint 3

Main 5 Discussion Index

Modeling Network Capacity

A link is valid only in one of three cases:

1. Coordinator $\rightarrow$ a node that holds the first layer. Payload is a token (Fig. 2 uses 4 bytes).
2. A node that holds the last layer $\rightarrow$ coordinator. Payload is again a token.
3. Node $i \rightarrow$ node j when j holds the layers needed immediately after i. Payload is an activation (Fig. 2 uses 16 KB).

$$\text{capacity (tokens/s)} = \frac{\text{profiled average bandwidth}}{\text{bytes per token or activation}}.$$

The formula is the paper's rule. The byte sizes above are the Fig. 2 annotations, not universal constants.

Activations cost far more tokens of budget than the final output token, so wide-area links saturate first.

Source: Helix, Sec. 4.3. Bandwidth is a one-time average, not a runtime measurement.

Main 5

Discussion Index

Why Max Flow Represents Serving Throughput

- One unit of flow is one token that can pass a node or a link in one second.
- Conservation forces tokens into c^{in} to leave through c^{out}.
- Helix computes the source–sink max flow with preflow-push. That value is the placement's max serving throughput.
- The model is a steady token rate. It does not output prompt latency, decode latency, or a percentile.
- Early stopping uses aggregate compute capacity normalized by total model layers (token-layer work/s divided by layers).

Source: Helix, Sec. 4.3 and the early-stop remark in Sec. 4.5.

Main 5

Discussion Index

Worked Three-Node Example

Placement

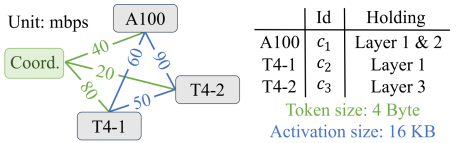
A100: layers 1–2.
T4-1: layer 1. T4-2: layer 3.

Activation link

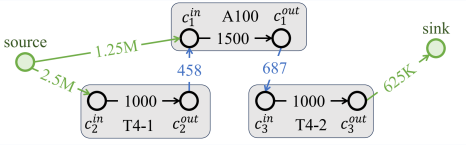
$$\frac{60 \times 10^6 \text{ bits/s}}{16 \times 1024 \times 8 \text{ bits/token}} \approx 458 \text{ tokens/s}$$

Interpretation

The KiB interpretation matches Fig. 2's rounded capacities. These payload sizes are example values.



(a) Physical links and layer placement



(b) Capacities in tokens/s

Activation transfers constrain the flow even when token-sized links are fast.

Source: Helix, Sec. 4.3, Fig. 2; unit conversion is illustrative.

Main 5 Graph Compute Network Throughput

Discussion Index

MILP Variables and Objective

Symbol	Role	Meaning
s_i	integer	first layer on node i
b_{ij}	binary	node i holds exactly j layers
$f_{i,j}$	real	flow from i to j, in tokens/s
$d_{i,j}$	binary	whether link (i, j) is valid
cond	binary	auxiliaries that linearize overlap

$e_i = s_i + \sum_j j\, b_{ij}$. Node i holds the half-open range $[s_i, e_i)$. k is the most layers that fit on that GPU.

$$\max \sum_i f_{\text{source},i}$$

The objective maximizes coordinator outflow. Table 5 groups the variable counts as linear in $|C|$ or $|E|$.

Source: Helix, Sec. 4.4, Tables 5–6. Notation here is compacted from the paper.

Main 6 Main 3 Placement Flow / Compute Network Discussion Index

Placement and Memory Constraints

Constraint 1: contiguous placement within the memory budget.

$$\sum_{j=1}^{k} b_{ij} = 1, \qquad 0 \le s_i < L, \qquad e_i \le L.$$

- Exactly one layer-count is chosen. The range must lie inside the model's L layers.
- Memory enters as k, the maximum number of layers the GPU can hold. Table 6 has no KV-byte variable.
- If $s_j < e_i < e_j$, traffic from i runs only layers $[e_i, e_j)$ on j. The paper calls this partial inference.
- The solver may be run with partial inference disabled. Then a link is valid only when $e_i = s_j$.

GPU memory limits how many layers are placed. KV capacity is enforced later, at schedule time.

Source: Helix, Sec. 4.4, Constraint 1, and the partial-inference remark.

Main 6 Discussion Index

Flow and Compute Constraints

Constraint 2: conservation at node i.

$$\sum_{u} f_{u,i} = \sum_{v} f_{i,v}.$$

Constraint 3: profiled inference throughput.

$$\sum_{u} f_{u,i} \leq \sum_{j=1}^{k} b_{ij}\, T_j.$$

- T_j is a profiled constant: tokens per second when this node holds j layers.
- The binaries b_{ij} pick exactly one of those profiled rates.
- Flow is still also limited by the network edges on the next slide.

Source: Helix, Sec. 4.4, Constraints 2–3. Sums skip node i itself, as stated in the paper.

Main 6

Discussion Index

Connectivity and Network Constraints

Constraint 4, compacted. A coordinator link into i exists only if i holds the first layer:

$$s_i \le L\,(1 - d_{\mathrm{source},i}). \qquad L\,d_{i,\mathrm{sink}} \le e_i.$$

Between compute nodes, $d_{ij} = 1$ only if $s_j \le e_i < e_j$. Two auxiliary binaries linearize that test, and $d_{ij} \le 0.5\,cond^1 + 0.5\,cond^2$.

Full Linearization

Constraint 5: transmission throughput.

$$f_{ij} \le d_{ij}\,S_{ij}.$$

S_{ij} is the profiled token rate of that link, from Sec. 4.3. Invalid links carry zero flow.

Source: Helix, Sec. 4.4, Constraints 4–5. The two-auxiliary form is the paper's linearization.

Main 6

Discussion Index

Full Connectivity Linearization

Binary $a_{ij}, b_{ij}^{\text{aux}}$ stand for the paper's $cond^1, cond^2$.

$$
\begin{aligned}
(L+1)(1-a_{ij}) &\geq s_j - e_i, \\
e_j - e_i &\geq 1 - (L+1)(1-b_{ij}^{\text{aux}}), \\
d_{ij} &\leq \tfrac{1}{2} a_{ij} + \tfrac{1}{2} b_{ij}^{\text{aux}}, \\
0 \leq f_{ij} &\leq d_{ij} S_{ij}.
\end{aligned}
$$

An active edge requires $s_j \leq e_i < e_j$. With no partial inference, require $e_i = s_j$ instead.

Interpretation: a valid connection need not carry positive flow. Link capacity bounds flow; it is not the IWRR weight.

Source: Helix, Sec. 4.4, Table 6. Auxiliary names adapted.

Main 6 Discussion Index

Making the MILP Practical

Pruning Drop slow network connections. Evaluation uses average degree 12. This restricts the search space.

Warm start Petals, Swarm, and SP supply feasible starting candidates; Gurobi searches for better placements.

Early stopping Stop near the analytic compute upper bound, or after 10 minutes without improvement. Maximum search budget: 4 hours on a 14-core CPU.

Why not relax to LP? Rounding start layers or layer counts can invalidate edges and radically change feasible flow.

Scaling beyond the evaluation Partitioning hundreds of nodes into smaller clusters is suggested, but not evaluated.

Source: Helix, Secs. 4.5–4.6, 6.2

Main 6 | Pruning / Warm Start | Solver Quality | Discussion Index

Runtime Components and Control Flow

1. **Once per cluster:** profile nodes and links, solve the MILP, load each node's layer range.
2. **On request arrival:** the coordinator's scheduler builds one pipeline and sends the request to its first node.
3. **On each node:** run the assigned layers for the requests that arrived during the previous batch. Do not add a waiting window.
4. **On the last node:** return the output token to the coordinator.
5. **On the next token:** reuse that request's pipeline. Do not solve the MILP again.

Source: Helix, Sec. 5 and Fig. 3. Batching details are in Sec. 5.1.

Main 4 | System Overview | Pipelines | Discussion Index

Per-Request Pipelines

- A valid pipeline runs every layer once, in order. Each stage names a node and the layers it will run.
- Fixed pipelines give each pipeline a disjoint set of machines.
- Helix lets pipelines intersect. The candidates are the source–sink paths of the placement graph.
- After the pipeline is chosen, a node batches every request that arrived while it ran the previous batch. There is no extra collect window.
- Later tokens of the same request keep that pipeline.

Source: Helix, Sec. 5.1. Fixed-pipeline contrast cites prior systems, including HexGen in the related work.

Main 7

Discussion Index

Converting Max-Flow Weights into Scheduling

Per-vertex state The coordinator maintains an IWRR scheduler for each vertex in the placement graph.

Candidates Neighbors reachable through valid directed edges. KV masking removes nodes above the estimated high-water mark.

Weights Use the optimized flow f_{uv} on each outgoing edge. Selection frequency is proportional to that weight.

Walk Choose the first worker from the coordinator's state, then choose successors until the path reaches the sink.

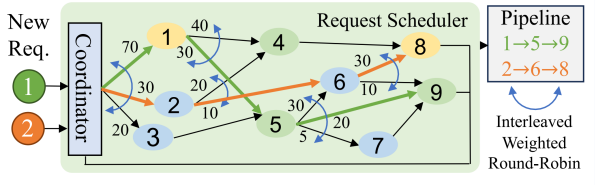
Interleave and reuse Interleaving avoids bursts. Subsequent tokens retain the selected pipeline and its KV state.

Source: Helix, Secs. 5.1–5.2, Fig. 4

Main 7 Pipelines Example KV Cache Discussion Index

Pipeline Scheduling Example

- Vertices are compute nodes. An edge exists only if the MILP placement made it valid.
- The number on an edge is flow in the max-flow solution, not link capacity.
- The figure's two example pipelines are $1 \to 5 \to 9$ and $2 \to 6 \to 8$.
- Those paths can share the cluster. They are not two frozen replicas.



Source: Helix, Sec. 5.1, Fig. 4. PDF p. 8.

Main 7

Discussion Index

KV-Cache Masking and Backpressure

- Each GPU has a finite VRAM budget for KV cache. Exceeding it forces offload to host memory, which the paper says significantly cuts throughput.
- Output length is unknown until generation ends, so the scheduler cannot know the exact KV footprint.
- Helix estimates each node's KV use with the average output length.
- A node above the high-water mark is masked out of IWRR until some of its current requests finish.
- The paper does not print the numeric threshold or the byte formula for that estimate.

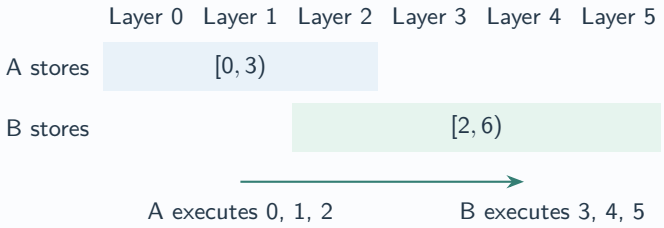
Backpressure is a mask on the same IWRR walk. It is not a second placement solver.

Source: Helix, Sec. 5.2. "High water mark" is the paper's term; its value is not specified.

Main 8 Main 4 Main 7 Partial Inference Discussion Index

Partial Inference and Request-Specific KV

Illustrative example: six layers, numbered 0–5. Half-open ranges.



Overlapping weights B stores layer 2, but this request skips it on B.

KV ownership Keep request-specific KV for the layers actually executed on each worker.

Valid hop: $s_B \leq e_A < e_B$. Strict overlap: $s_B < e_A < e_B$.
B runs $[e_A, e_B)$ for this request. Other paths may require different layers on B.
vLLM's unified page pool supports partial inference; later tokens reuse the route.

Source: Helix, Secs. 4.4, 5.1–5.2, 6.1; illustrative example

Main 8

Discussion Index

Implementation and Prototype

Prototype vLLM 0.4.0.post1, 1.5k Python + 1.7k C++ lines. Unified page pool for partial inference, ZeroMQ communication, and Gurobi placement.

Simulator 14k Python lines model heterogeneous GPUs and networks. Geo-distributed and 42-node results use this simulator.

Validation scope Against the single-cluster prototype: average decode-throughput error $< 5\%$, prompt-latency error $< 5\%$, decode-latency error $< 4\%$.

Hardware scope Prototype experiments use single-GPU nodes. Multi-GPU nodes use in-node TP; the 42-node simulation includes them.

Artifact scope Appendix A describes dummy weights. The artifact validates system performance, not generated-answer quality.

Source: Helix, Secs. 6.1–6.3, Appendix A.2 and A.3.5

Main 9 Discussion Index

Evaluation Methodology

Setup	Hardware	Network	Evidence
Single	4 A100 + 8 L4 + 12 T4	10 Gb/s	Prototype
Geo	Same GPUs in 3 clusters	100 Mb/s, 50 ms	Simulation
42-node	7 types, single / multi-GPU nodes	Simulated	Simulation

- Models: LLaMA-1 30B and LLaMA-2 70B, FP16. Baselines: Swarm, and separate pipelines (SP) inside Helix.
- Petals is compared only as a placement (Sec. 6.6), not as an end-to-end server.
- Offline metric: average decode throughput, in generated tokens/s. Online adds average prompt and decode latency.
- Online arrivals are the Azure shape scaled to 75% of that cluster's peak. Offline arrivals saturate the cluster.

Source: Helix, Sec. 6.2. The 100 Mb/s geo setting is justified by Table 7, whose pairs span 54–204 Mb/s.

Main 9 Implementation Baselines Workload Discussion Index

Hardware, Baselines, and Normalization

42 nodes: 4 A100, 6 V100, 8 L4, 10 T4, 4 2×L4, 6 2×T4, 4 4×T4.

Geo clusters: (4 A100), (2 L4 + 8 T4), (6 L4 + 4 T4). Measured cross-region links in Table 7 span 54–204 Mb/s; experiments simulate mean 100 Mb/s / 50 ms.

Swarm: placement and scheduler reimplemented in Helix; minimum stage count lets the weakest GPU fit one stage in half its VRAM.

SP: one replica per GPU type, implemented in Helix. **Petals:** placement only, using Helix's scheduler.

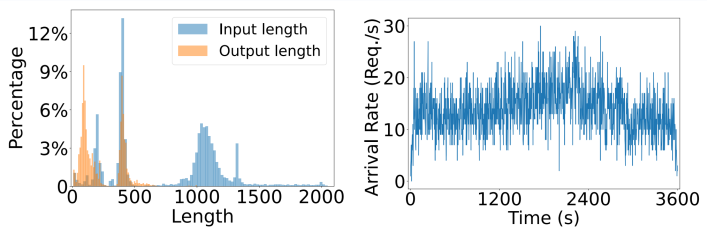
Ratios: $x\times = \text{Helix throughput}/\text{named baseline throughput}$. Percentage latency change uses that baseline's mean latency.

Fig. 11: throughput and solver-time comparisons use the paired without-pruning / default-initialization run as the reference, respectively. Fig. 12 plots an objective value and bound, not a serving-latency speedup.

Source: Helix, Secs. 6.2, 6.5–6.9, Table 7

Main 10 Discussion Index

Workload Characteristics



Filtering: remove input > 2048 or output > 1024 tokens; retain 16,657 requests.

Mean lengths: 763 input tokens and 232 output tokens.

Runs: online 30 s warmup + 30 min; offline 1 min warmup + 10 min.

Figure: Azure Conversation before online 75% scaling. Latency results apply to this filtered trace.

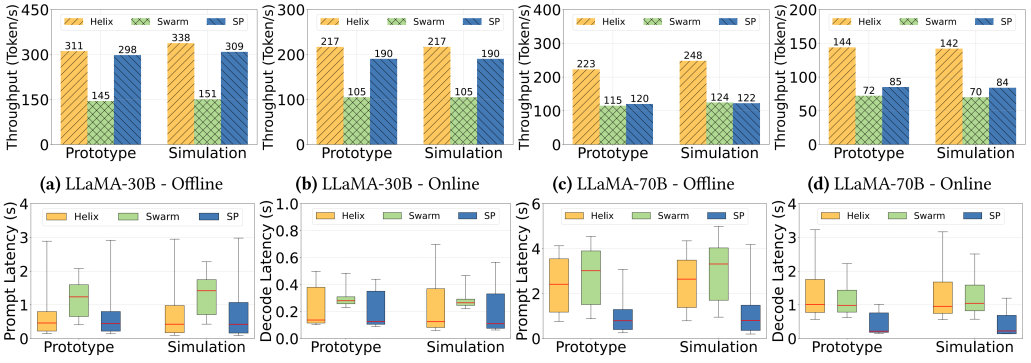
Source: Helix, Sec. 6.2, Fig. 5. PDF p. 8.

Main 9

Discussion Index

Complete Figure 6: Single-Cluster Results

24-node prototype and simulator check. FP16 30B / 70B. Baselines: SP and Swarm.



Top: decode tokens/s. Bottom: 30B prompt/decode, then 70B prompt/decode (s).
Boxes: 25–75th percentiles; whiskers: 5–95th; red: median.

Source: Helix, Sec. 6.3, Fig. 6

Main 11 Detailed Ratios

Discussion Index

Single-Cluster Results

Fig. 6. Prototype. (a–d) decode throughput; (e–h) online latency. Box 25–75, whisker 5–95, red line = median.

Model	Versus	Offline tput	Online tput	Prompt lat.	Decode lat.
30B	SP	+4%	+14%	−2%	+10%
30B	Swarm	2.14×	2.07×	−32%	−12%
70B	SP	1.86×	1.69×	average latency higher	
70B	Swarm	1.94×	2.00×	−15%	+16%

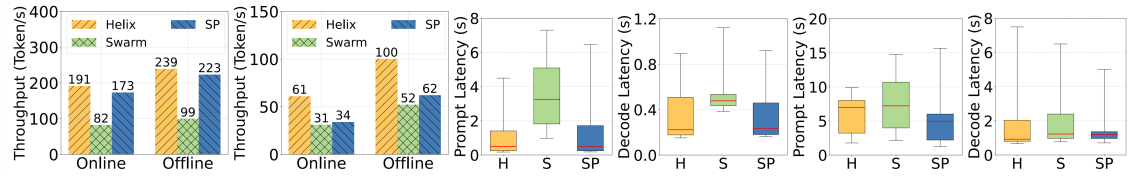
30B: three replicas, one GPU type each. 70B vs. SP: latency is higher because SP serves most requests on A100s and leaves L4/T4 idle.

Source: Helix, Sec. 6.3, Fig. 6. Pairs are offline then online. PDF p. 9.

Main 11 Full Figure Discussion Index

Complete Figure 7: Geo-Distributed Results

Simulation: 24 nodes in 3 clusters; mean inter-cluster 100 Mb/s and 50 ms.



Left: 30B / 70B tokens/s. Right: 30B prompt/decode, then 70B prompt/decode (s).
H = Helix, S = Swarm; boxes 25–75th, whiskers 5–95th, red = median.

Simulated 30B vs. Swarm Online prompt latency −66%, decode latency −24%.

70B vs. SP Decode throughput: 1.61× offline and 1.79× online.

Mechanism Shallower pipelines reduce slow inter-region transfers.

Source: Helix, Sec. 6.4, Fig. 7

Main 12 Detailed Ratios

Discussion Index

Geo-Distributed Results

Simulation. Throughput pairs: offline / online; latency changes: online means.

Model	Versus	Throughput	Prompt latency	Decode latency
30B	SP	+7% / +10%	−14%	+2%
30B	Swarm	2.41× / 2.33×*	−66%	−24%
70B	SP	1.61× / 1.79×	—	—
70B	Swarm	1.92× / 1.97×	−21%	−7%

*Fig. 7 identifies offline / online; Sec. 6.4 lists the two ratios without ordering them.

70B placement: pipelines are 28% shallower than Swarm and 19% shallower than Helix's fast-network plan.

Offline prompt example: Swarm takes 71 s, 7.5× Helix.

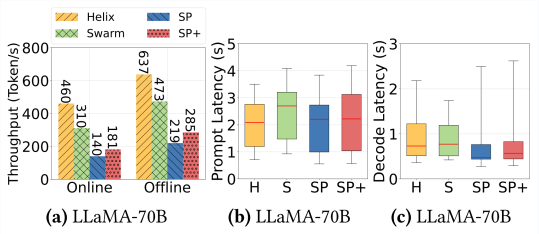
Abstract claim: the 66% / 24% latency reductions refer to 30B online vs. Swarm.

Source: Helix, Sec. 6.4, Fig. 7. Latency dashes mean not tabulated here.

Main 12 Full Figure

Discussion Index

High GPU-Heterogeneity Results



(a) LLaMA-70B **(b)** LLaMA-70B **(c)** LLaMA-70B

Helix versus	Swarm	SP	SP+
Offline throughput	1.37×	2.91×	2.24×
Online throughput	1.48×	3.29×	2.54×
Online prompt latency	−17%	−1%	−7%

42-node simulation, 70B. SP omits V100 / T4 / 2×T4 nodes, which cannot form separate pipelines; SP+ combines them. Helix uses slower GPUs and has slightly higher decode latency. The abstract rounds 3.29× to 3.3×.

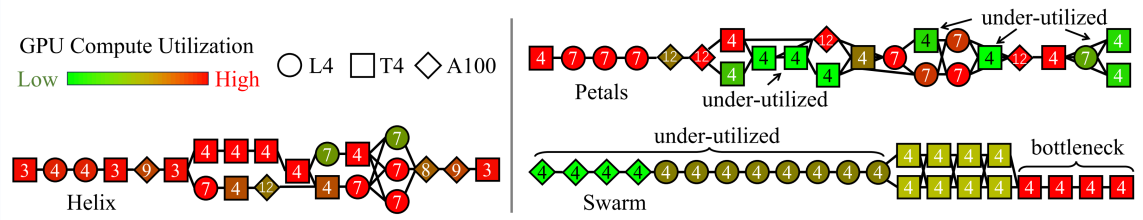
Source: Helix, Sec. 6.5, Fig. 8. Simulation, not a prototype measurement.

Main 13

Discussion Index

Figure 9: Placement Graph Detail

Single-cluster GPU compute utilization. Same Helix scheduler; different placements.



Node numbers are layer counts; color shows GPU compute utilization.

Swarm's tail bottleneck leaves faster GPUs under-used. Helix keeps almost every node busy.

Source: Helix, Sec. 6.6, Fig. 9(b), enlarged crop. Full figure on Main 14.

Main 14 Detailed Ratios

Discussion Index

Model Placement Deep Dive

Fig. 9. Offline LLaMA 70B. Every method uses Helix's scheduler. Integers in (b) are layer counts.

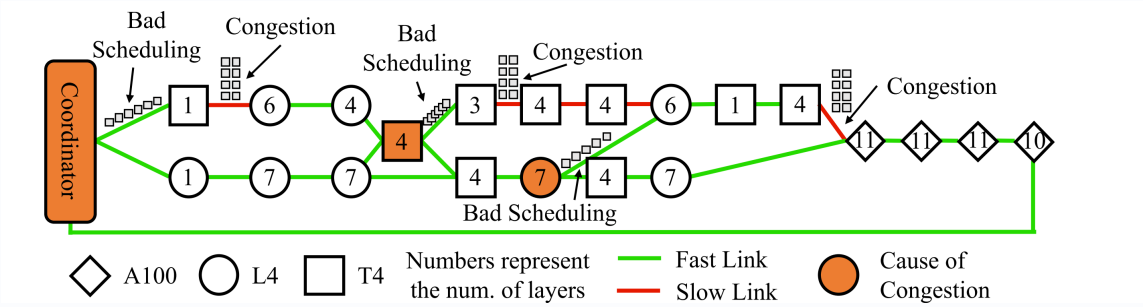
- Vs. Petals / Swarm: 1.23× / 2.10× on the single cluster, and 1.49× / 2.38× on the geo-distributed simulator.
- Single-cluster case: Swarm's tail is four T4s with four layers each, so A100 and L4 stay under-used.
- Petals leaves eight T4s and one L4 under-used. Helix's plan keeps almost every node busy.

Source: Helix, Sec. 6.6, Fig. 9. PDF p. 11. Petals is not an end-to-end baseline.

Main 14 Main 10 Graph Detail Discussion Index

Figure 10: Scheduling Graph Detail

Geo-distributed case study. Same Helix placement; scheduling can congest slow links.



Orange nodes mark scheduling decisions that cause congestion.

Prompt traffic waits 5–16 s on three slow links; one queue is caused three hops upstream.

Source: Helix, Sec. 6.7, Fig. 10(b), enlarged crop. Full figure on Main 15.

Main 15 Detailed Ratios Discussion Index

Request Scheduling Deep Dive

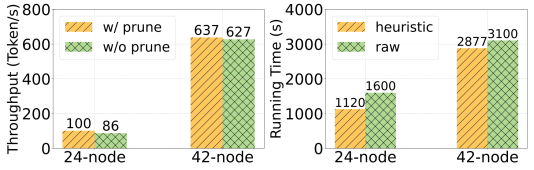
Fig. 10. Offline LLaMA 70B. Every method uses Helix's placement. (b) is the geo-distributed case study.

- Vs. Swarm / random: +30% / +29% on the single cluster, and +22% / +15% on the geo-distributed clusters.
- Vs. shortest queue, geo-distributed only: +19% decode throughput.
- In (b), prompt traffic waits 5–16 s on three slow links. One of those queues is caused by a scheduler three hops away.

Source: Helix, Sec. 6.7, Fig. 10. PDF p. 12.

Main 15 Graph Detail Discussion Index

MILP Optimization Ablation



(a) Cluster pruning. **(b)** Initial values.

Offline 70B	24-node geo	42-node heterogeneous
Variables / constraints, pruned	876 / 1122	2144 / 2772
Variables / constraints, unpruned	1376 / 1848	4004 / 5502
Links removed / problem-size reduction	50% / 36%	72% / 46%
Throughput gain with pruning	16%	2%
Warm-start time saved (bars)	30%	7.2%

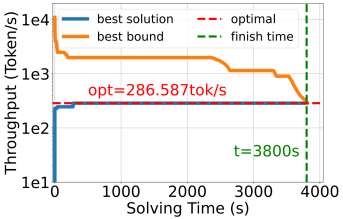
Degree-12 pruning eases the timed search; it does not prove deleted links useless. Warm start finds the same placement, with 1.43× / 1.08× solver speedup.

Source: Helix, Sec. 6.8, Fig. 11, Table 8. Text says 43% / 8% less time; bars imply 30% / 7.2%.

Main 6 | Solver Settings | Solver Quality | Discussion Index

Solver Time and Solution Quality

- Instance: LLaMA 30B on 4 L4s and 6 T4s. This is smaller than the 24- and 42-node runs.
- The best feasible placement reaches the optimum in under 5 minutes.
- Certifying that optimum takes more than an hour of branch-and-bound.
- The red dotted line is that cluster's optimal throughput. The upper bound tightens as cuts are added.



Early stop is reasonable here. Fig. 12 does not certify the 42-node plan.

Source: Helix, Sec. 6.9, Fig. 12. PDF p. 13.

Main 6

Discussion Index

Limitations and Threats to Validity

- Placement is computed once. A workload shift does not trigger a new MILP in the described runtime.
- Geo-distributed and 42-node numbers are simulated. The $< 5\%$ error is a single-cluster prototype check.
- The KV mask uses average output length. The paper does not specify the high-water mark.
- Pruning and the 10-minute stall rule mean a deployed plan need not be a proven optimum.
- Decode latency can rise when Helix actually uses the slow GPUs that SP leaves idle.
- Evidence is FP16 LLaMA-1 30B and LLaMA-2 70B on one pruned Azure trace. The conclusion's "guaranteed optimal" is the MILP's claim, not every timed run.

Source: Helix, Sec. 4.5, Sec. 6, and Sec. 8. The last bullet contrasts the conclusion with Sec. 6.2 and Sec. 6.9.

Main 16

Discussion Index

Research Directions and Applicability

Static placement How much throughput gain justifies weight movement and KV migration?

KV estimation Could output-length uncertainty guide admission without wasting VRAM?

Latency trade-offs Can SLO-constrained goodput replace pure throughput while retaining a tractable model?

Workload sensitivity Re-test different prompt/decode mixes, burstiness, bandwidth variation, and failures.

Evidence gap Validate geographically distributed execution and larger heterogeneous clusters on hardware.

Source: Helix, Interpretation / research extensions motivated by Helix, Secs. 4.5, 5.2, 6.2–6.9. Not implemented by the paper.

Main 16

Discussion Index